

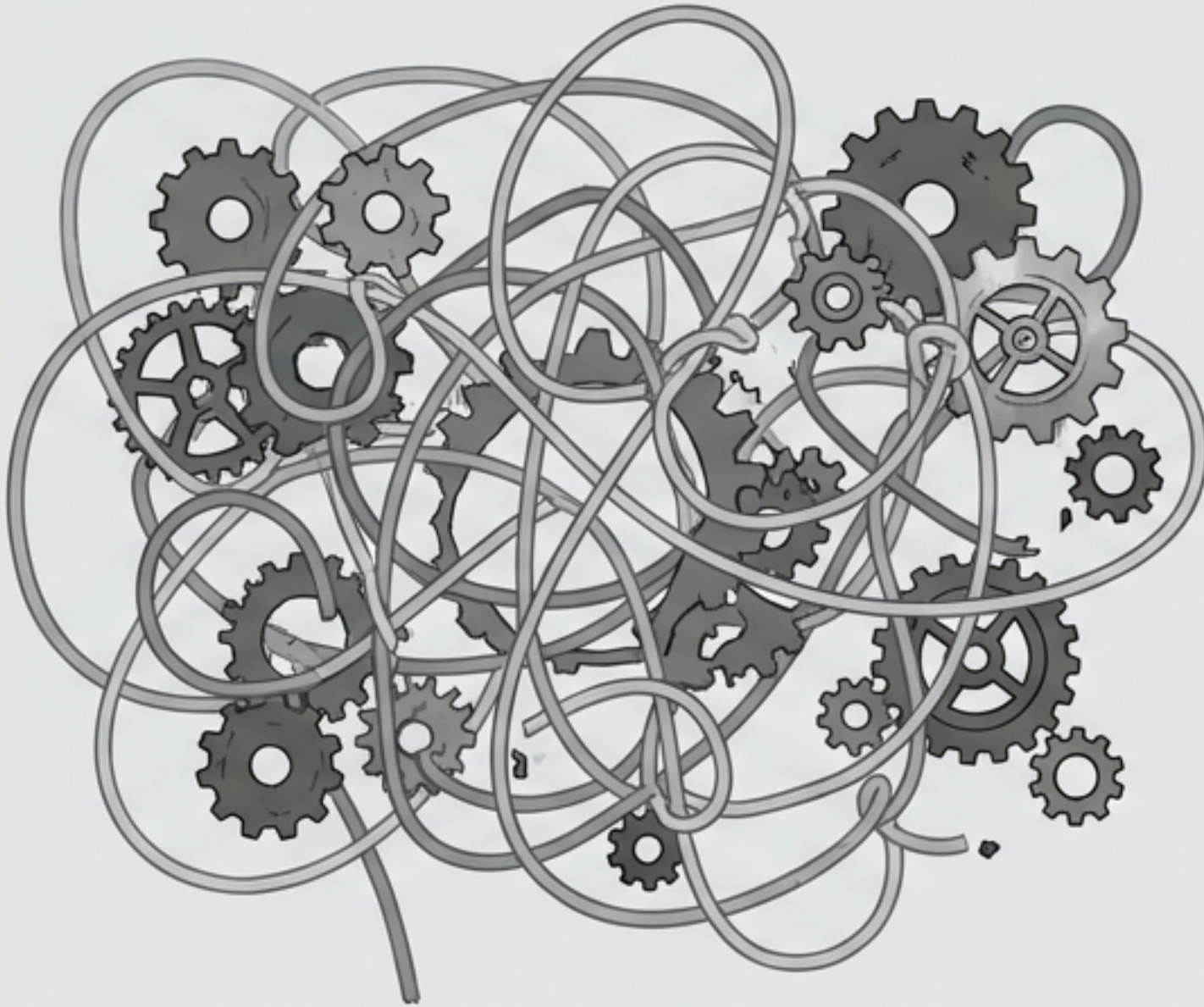


Aula 1: Introdução à Orientada a Objetos

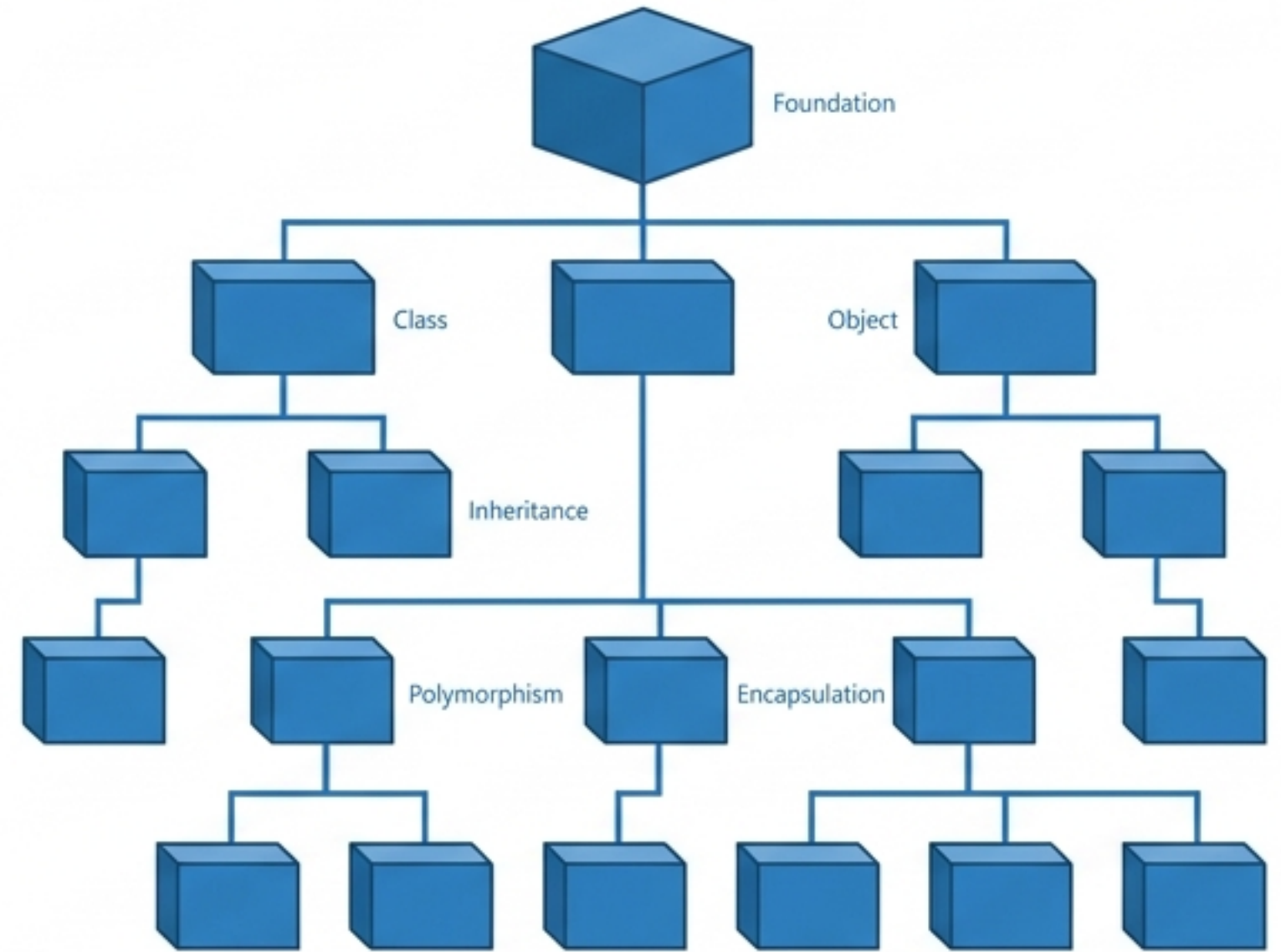
Prof. Ronaldo Borges

A Crise da Complexidade

Programação Estruturada

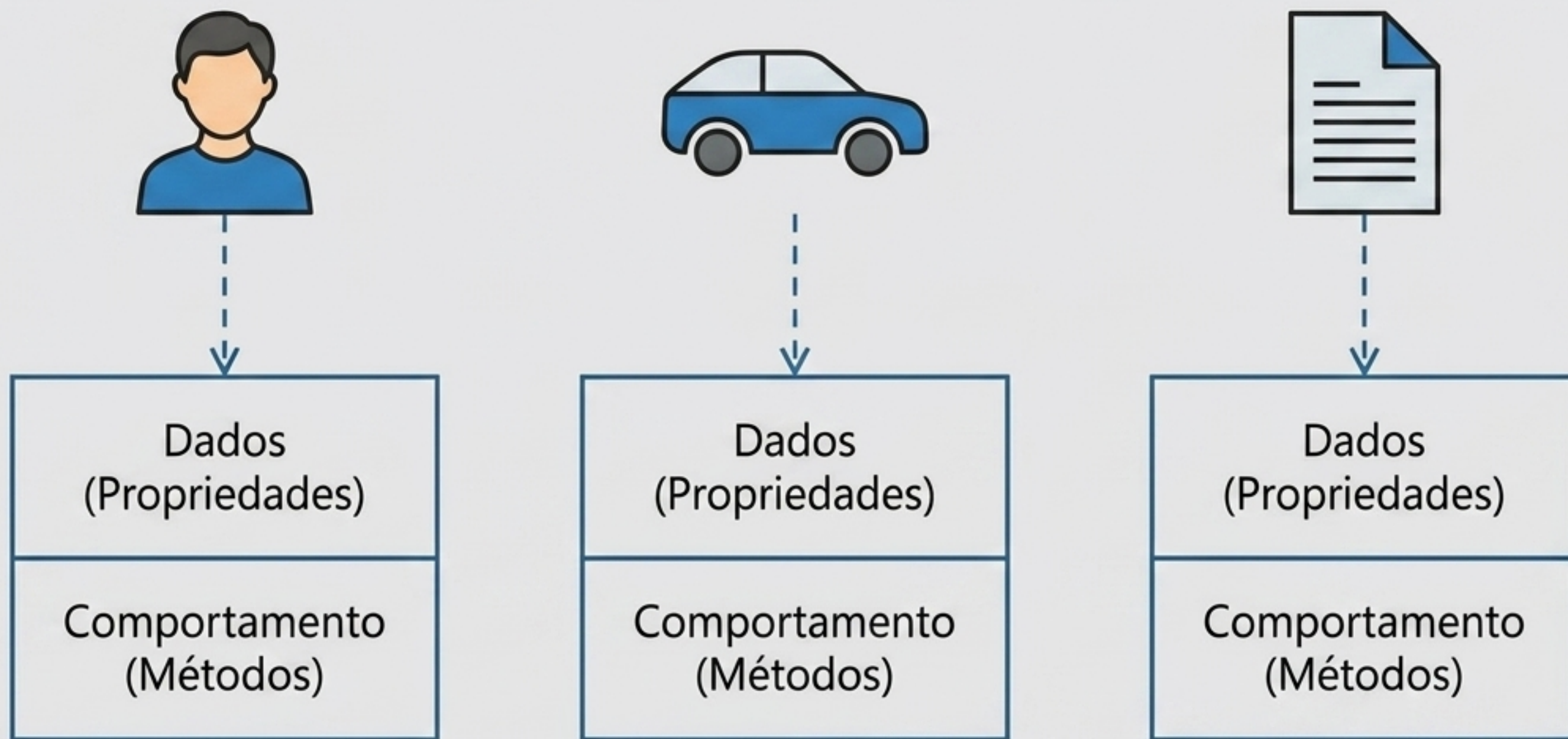


Orientação a Objetos



O software moderno cresceu. Paradigmas procedurais geravam códigos emaranhados, difíceis de manter e escalar. A solução exigia uma nova forma de modelar o mundo.

O Paradigma da Orientação a Objetos



O foco muda das rotinas para as entidades. O software passa a ser uma coleção de objetos independentes que se comunicam entre si, refletindo o mundo real.

Metáfora Visual: A Planta e a Casa

Classe (O Molde)



Define a estrutura, os atributos e o comportamento. É apenas um conceito abstrato.

Objetos (As Instâncias)



A manifestação física e real da classe na memória. Cada objeto possui seu próprio estado.

O Desafio do JavaScript Moderno



```
let idade = 25;
```

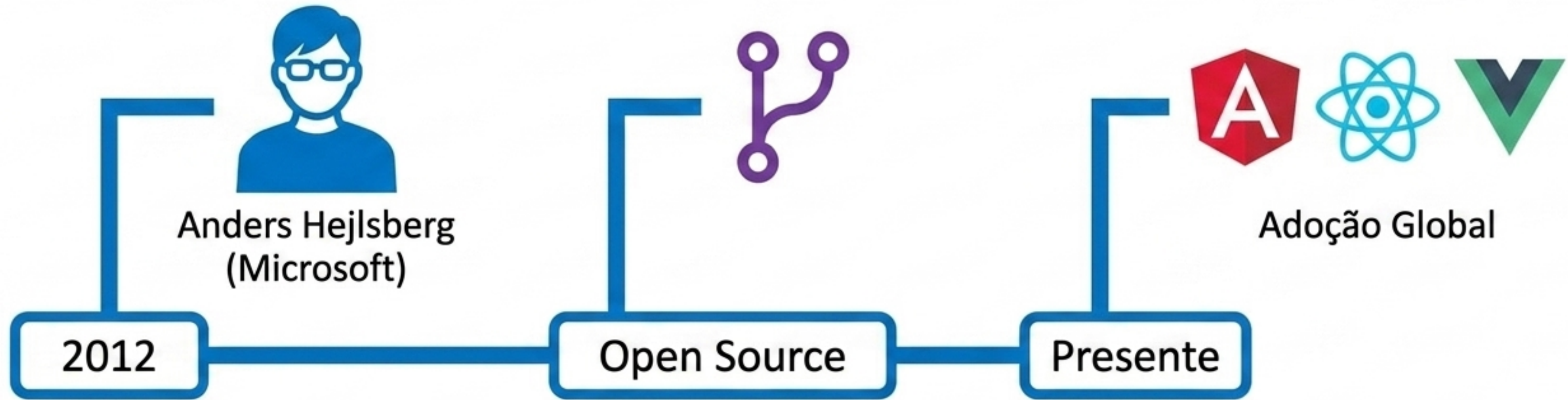
```
idade = 'Vinte e cinco'; // Nenhum erro no editor
```

Runtime Error
(Falha em Produção)



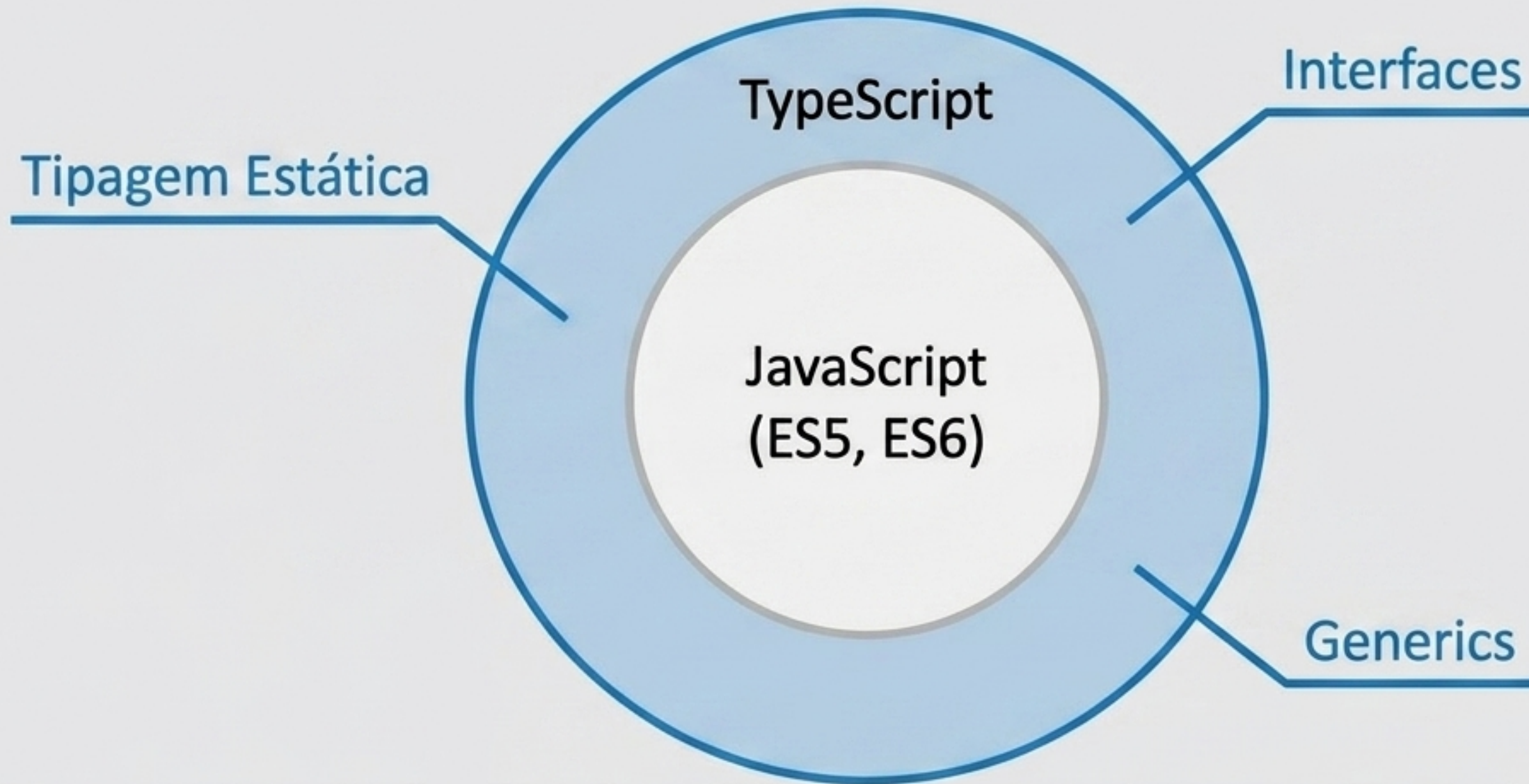
Criado inicialmente para interações web simples, o JS possui tipagem dinâmica. Em grandes projetos de POO, isso permite que erros silenciosos entrem em produção, tornando a manutenção caótica.

O Surgimento do TypeScript



Motivação: trazer a disciplina da tipagem estática e as estruturas sólidas de POO para a escala global do JavaScript.

TypeScript é um Superset



Ele não substitui o JS; ele adiciona superpoderes ao JS. Todo código JavaScript válido já é, por definição, um código TypeScript válido.

O Processo de Transpilação

Syntesis: Browsers não executam TypeScript nativamente. O código é verificado durante o desenvolvimento e convertido em JavaScript puro e seguro para produção.



Matriz Diagnóstica: JS vs TS

Característica	JavaScript	TypeScript
Tipagem	Dinâmica (Fraca)	Estática (Forte)
Captura de Erros	Em Execução (Runtime)	Na Compilação (Editor)
Escala Ideal	Scripts / Pequena	Enterprise / Larga escala

O Superpoder da Tipagem Estática

JavaScript: Falha Silenciosa

```
function soma(a, b) {  
    return a + b;  
}  
  
let resultado = soma("1", 2);  
console.log(resultado); // "12"
```

TypeScript: Alerta Imediato

```
function soma(a: number, b: number) {  
    return a + b;  
}
```

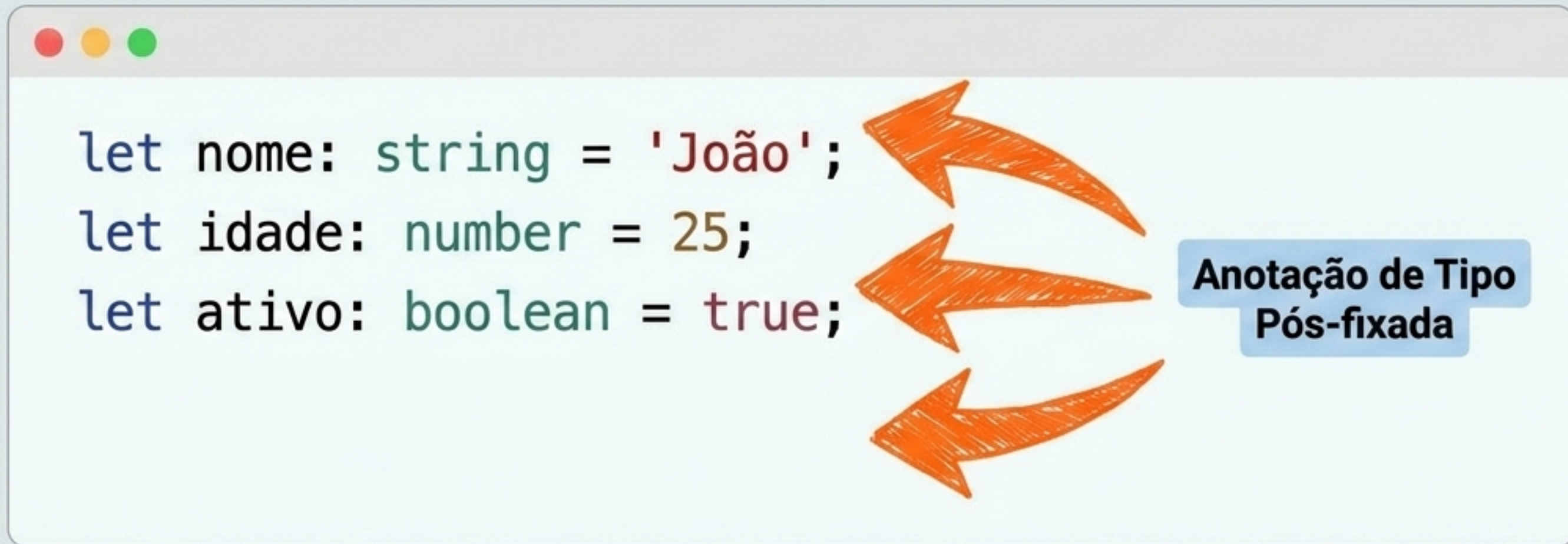
```
let resultado = soma("1", 2);
```

Argument of type 'string' is not assignable to parameter of type 'number'

Synthesis: O TS permite descobrir e corrigir o erro imediatamente no seu editor, não no console do navegador do seu usuário final.

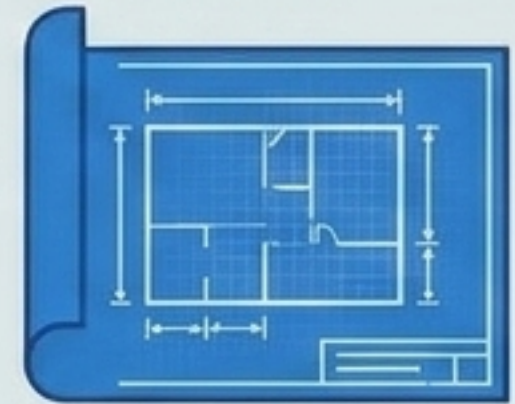
Exemplo Prático 1: Variáveis e Tipos Primitivos

Synthesis: A sintaxe base é idêntica ao ES6, mas adicionamos anotações explícitas de tipo para impor o contrato de dados.



Exemplo Prático 2: Classes e Objetos

```
class Pessoa {  
    nome: string;  
    constructor(nome: string) {  
        this.nome = nome;  
    }  
    apresentar() { return 'Olá, sou ' + this.nome; }  
}  
  
let usuario = new Pessoa('Ana');
```



A Planta



A Casa

Exemplo Prático 3: Interfaces (O Contrato)

Synthesis: Interfaces não existem no JavaScript compilado. Elas são ferramentas exclusivas de design no TS para garantir que um objeto obedeça perfeitamente a uma forma antes do software rodar.

```
interface IPessoa {  
    nome: string;  
    idade: number;  
}  
  
function registrar(p: IPessoa) {  
    // ...  
}
```

Contrato Estrutural Obrigatório



Síntese: Por que TypeScript para POO?

O TypeScript fornece o rigor, a modelagem de contratos e as ferramentas de checagem exigidas para implementar a verdadeira Orientação a Objetos na Web de forma previsível.



Referências Bibliográficas

LEITE, T. et al. Orientação a objetos: aprenda seus conceitos e suas aplicabilidades de forma efetiva. Casa do Código, 2016.

TURINI, R. Desbravando Java e Orientação a Objetos: Um guia para o iniciante da linguagem. Casa do Código, 2014.

ADRIANO, S. A. Guia prático de TypeScript. Casa do Código, 2021.

FURGERI, S. Programação orientada a objetos: Conceitos e técnicas. Saraiva, 2015.

UZAYR, S. B. TypeScript for Beginners: The Ultimate Guide. CRC Press, 2022.